

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 02 -

MLOPS FUNDAMENTALS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMSE

O QUE VEM POR AÍ?

Nesta aula vamos aprender um pouco sobre como um computador funciona por “debaixo dos panos”. Vamos conversar sobre Linux, bash programming e processos. Esses conceitos serão úteis para você, independente da área de programação que você queira atuar no futuro.



HANDS ON

O conteúdo dessa aula é importante para qualquer carreira próxima do desenvolvimento de software. Nessa aula, vamos aprender como nos comunicar com o nosso computador sem precisar terceirizar outros programas no caminho. Dessa forma, você se tornará um(a) profissional mais flexível para utilização das diversas ferramentas que já existem e aquelas que futuramente serão criadas.



SAIBA MAIS

Quando estamos trabalhando como engenheiro(a) de machine learning é recorrente a utilização de um terminal, mas o que é isso? Talvez você conheça esse termo como CMD (Command Prompt), a famosa tela preta dos programadores. O CMD é um terminal muito utilizado no SO (sistema operacional) Windows. O terminal nada mais é do que um programa que roda um shell, mas o que é um shell? O shell é uma interface de usuário que permite executar comandos que se comunicam com o kernel do SO. Kernel é o componente do SO responsável por permitir nossa comunicação com o hardware do computador.

Existem várias opções de terminais, shells e sistemas operacionais, cada um com seus pontos fortes e fracos. No caso dessa disciplina o mais importante é termos acesso ao shell bash, o mais utilizado no contexto de MLOps.

Para executar o shell bash basta abrir um terminal e executar o comando bash, o que provavelmente começará a executar o programa bash. Primeiramente, esse comando funciona porque o programa bash está no folder /bin/bash do seu computador, conforme mostra a Figura 1:



```
(ins)$ ls /bin/
[ dash expr ln pwd stty zsh
bash date hostname ls realpath sync
cat dd kill mkdir rm tcsh
chmod df ksh mv rmdir test
cp echo launchctl pax sh unlink
csh ed link ps sleep wait4path
```

Figura 1 - Folder /bin de um sistema operacional MacOS
Fonte: Elaborado pelo autor (2024)

Além disso, o folder /bin está na variável de ambiente **PATH**. Para ver, basta digitar o comando **echo \$PATH** no terminal (assista a videoaula para ver esse comando em execução). Podemos entender as variáveis de ambiente como um conjunto de chave-valor que os programas do nosso computador têm acesso.

Agora vamos aprender sobre o termo **Shebang**. Esse termo é utilizado para se referenciar a um tipo especial de comentário representado pelos caracteres **#!/**, o qual precisa estar na primeira linha do arquivo a ser executado. O Shell utiliza esse símbolo

para entender que precisa selecionar um interpretador para executar o resto do script, conforme mostra a Figura 2.

<pre>1 #!/bin/bash 2 3 echo "Hello World"</pre>	<pre>1 #!/usr/bin/python3 2 3 print('Hello World')</pre>
---	--

Figura 2 - Scripts “hello world” utilizando shebang para executar código bash (imagem à esquerda) e python (imagem à direita)

Fonte: Elaborado pelo autor (2024)

Quando um script contém a shebang `#!/bin/bash`, o denominamos como um script bash. Conforme vamos aprendendo mais comandos do sistema operacional, esses scripts vão tendo aplicações mais interessantes. Além disso, ao longo do aprendizado na carreira de dados você com certeza vai se deparar em algum momento com algum script desses. Vamos aprender sobre os comandos mais utilizados em terminais Linux:

- `ls` (list): lista os arquivos de um diretório;
- `du` (disk usage): mostra espaço em disco utilizado;
- `df` (disk free): mostra espaço em disco livre;
- `pwd` (print working directory): mostra o caminho do folder atual;
- `cd` (change directory): altera o caminho do folder atual;
- `which`: encontra o caminho de um executável (percorrendo a variável `PATH`);
- `grep`: procura padrões correspondentes;
- `echo`: escreve no standard output.

File related:

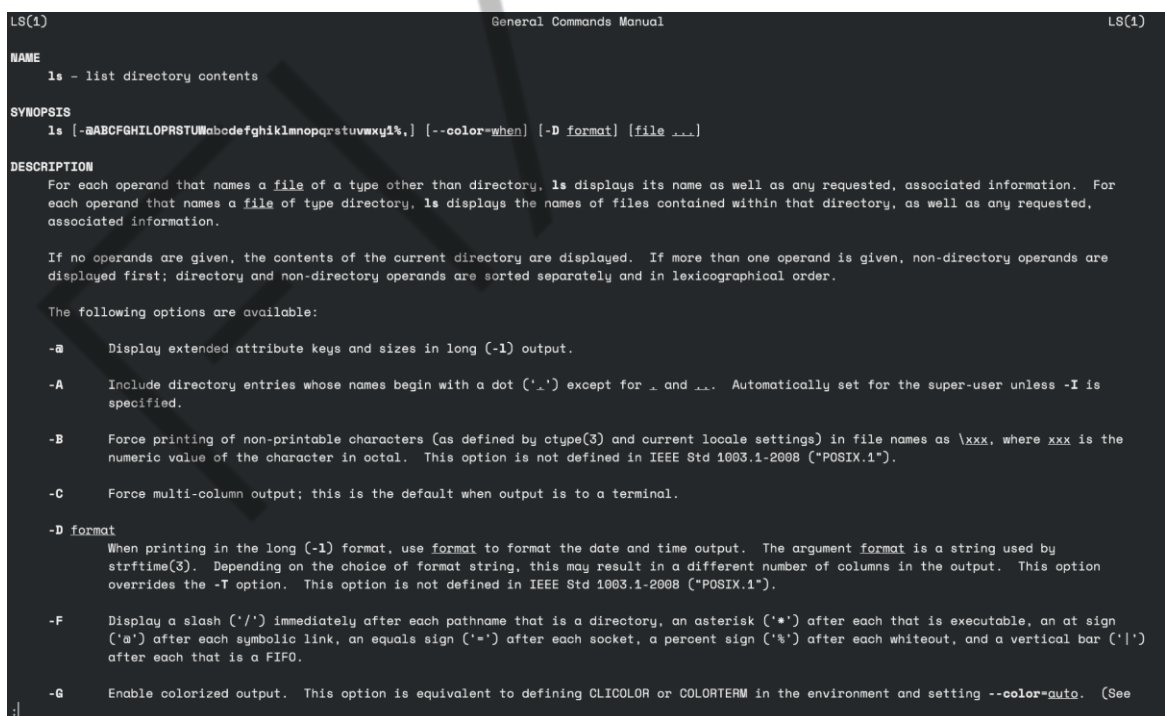
- `less`: mostra o conteúdo de um arquivo no terminal conforme solicitado pelo usuário;
- `cat`: mostra todo o conteúdo de um arquivo no terminal;
- `wc` (word count): exibe o número de linhas, palavras e bytes;

- touch: cria um arquivo vazio;
- mkdir (make directory): cria um diretório vazio;
- rm (remove): remove arquivos;
- chmod: altera as permissões de um arquivo;
- mv: move o arquivo para outro folder. Normalmente utilizado para mudar o nome de um arquivo.

Process related:

- ps (process status): lista os processos de uma máquina.
- fg (foreground): move processo em segundo plano (background) para o primeiro plano (foreground).
- kill: termina um processo.

Podemos ver a documentação dos comandos utilizando o comando **man**, conforme mostra a Figura 3:



```
LS(1) General Commands Manual LS(1)

NAME
  ls - list directory contents

SYNOPSIS
  ls [-aABCFGHILOPRSTUWabdefghiklmnopqrstuvwxy1%,] [--color=when] [-D format] [file ...]

DESCRIPTION
  For each operand that names a file of a type other than directory, ls displays its name as well as any requested, associated information. For each operand that names a file of type directory, ls displays the names of files contained within that directory, as well as any requested, associated information.

  If no operands are given, the contents of the current directory are displayed. If more than one operand is given, non-directory operands are displayed first; directory and non-directory operands are sorted separately and in lexicographical order.

  The following options are available:

  -a      Display extended attribute keys and sizes in long (-l) output.

  -A      Include directory entries whose names begin with a dot ('.') except for . and .. . Automatically set for the super-user unless -I is specified.

  -B      Force printing of non-printable characters (as defined by ctype(3) and current locale settings) in file names as \xxx, where xxx is the numeric value of the character in octal. This option is not defined in IEEE Std 1003.1-2008 ("POSIX.1").

  -C      Force multi-column output; this is the default when output is to a terminal.

  -D format
      When printing in the long (-l) format, use format to format the date and time output. The argument format is a string used by strftime(3). Depending on the choice of format string, this may result in a different number of columns in the output. This option overrides the -T option. This option is not defined in IEEE Std 1003.1-2008 ("POSIX.1").

  -F      Display a slash ('/') immediately after each pathname that is a directory, an asterisk ('*') after each that is executable, an at sign ('@') after each symbolic link, an equals sign ('=') after each socket, a percent sign ('%') after each whiteout, and a vertical bar ('|') after each that is a FIFO.

  -G      Enable colorized output. This option is equivalent to defining CLICOLOR or COLORTERM in the environment and setting --color=auto. (See
```

Figura 3 - Execução do comando man ls no terminal

Fonte: Elaborado pelo autor (2024)

Além disso, esses comandos obedecem o seguinte padrão: **command - options target**, conforme mostra a Figura 4 (assista a videoaula para ver alguns comandos e suas opções):

```
(ins)$ du -ah
4.0K    ./file1.txt
40K     ./folder/file2.txt
40K     ./folder
44K     .
```

Figura 4 - Execução do du -ah no terminal
Fonte: Elaborado pelo autor (2024)

Agora vamos aprender sobre operadores. Eles são muito úteis e interagem com comandos de alguma forma, como por exemplo o pipe, representado pelo símbolo: |. Ele serve para redirecionar o output de um comando para outro (em outras palavras o stdout de um comando para o stdin de outro comando). Por exemplo, suponha que a gente gostaria de listar quantos arquivos temos no diretório atual. O comando ls lista a quantidade de arquivos e o comando wc consegue contar, logo basta juntá-los utilizando o pipe, conforme mostra a Figura 5.

```
(ins)$ ls
file1.txt folder

(ins)$ ls . | wc -l
2
```

Figura 5 - Exemplo de utilização do pipe para direcionar o output de um comando para outro comando
Fonte: Elaborado pelo autor (2024)

Podemos redirecionar o output de um comando diretamente para um arquivo utilizando o operador >, conforme mostra a Figura 6.

```
(ins)$ echo "Vamos salvar esse texto em um arquivo" > file1.txt

(ins)$ cat file1.txt
Vamos salvar esse texto em um arquivo
```

Figura 6 - Exemplo de utilização do comando > para criar um arquivo
Fonte: Elaborado pelo autor (2024)

Caso o arquivo não exista, ele será criado. Entretanto, caso ele já exista, seu conteúdo será sobrescrito. Caso o objetivo seja fazer o append em um arquivo já existente pode-se utilizar o operador `>>`. Outro operador muito interessante para conhecermos é o `&&`. Ele funciona como um AND em programação, conforme mostra a Figura 7:

```
(ins)$ ls && cat file1.txt  
file1.txt folder mybash  
Vamos salvar esse texto em um arquivo
```

Figura 7 - Exemplo de utilização do comando `&&`
Fonte: Elaborado pelo autor (2024)

Se o comando anterior falhar, o comando seguinte não será executado, pois, utilizando a tabela **Verdade**, saberemos que se temos um Falso na operação AND, o resultado é **Falso**. Portanto, ele funciona como uma execução condicional. Por exemplo: executar um script somente se ele existir conforme mostra a Figura 8:

```
(ins)$ [ -f my_bash_script ] && ./my_bash_script  
Esse arquivo existe
```

Figura 8 - Exemplo de uso comum do operador `&&` para executar um arquivo somente se ele existir
Fonte: Elaborado pelo autor (2024)

Todos os comandos e operadores que vimos até agora utilizam o conceito de **standard input** (entrada do programa) e **standard output** (saída do programa). Porém, também precisamos entender o **standard error**. Quando acontece um erro nos programas, normalmente o direcionamos para um lugar para que sejam tratados. Suponha que você execute o comando `ls` no terminal para listar um diretório que não existe, conforme mostra Figura 9:

```
(ins)$ ls NOT_EXIST_FOLDER/  
"NOT_EXIST_FOLDER/": No such file or directory (os error 2)
```

Figura 9 - Listando um folder inexistente
Fonte: Elaborado pelo autor (2024)

Digitar o comando: `ls NOT_EXIST_FOLDER/ > error.txt` não redirecionará o output para o arquivo `error.txt`. Isso acontece porque a saída do programa foi para o

standard error, e para conseguirmos direcionar o erro precisamos utilizar o operador **2>**, conforme mostra a Figura 10.

```
(ins)$ ls folder 2> error.txt  
(ins)$ cat error.txt  
"folder": No such file or directory (os error 2)
```

Figura 10 - Redirecionando um erro para um arquivo
Fonte: Elaborado pelo autor (2024)

Quando não há interesse no erro, é usual a utilização do padrão **2>/dev/null**. Exemplo: **ls NOT_EXIST_FOLDER/ 2>/dev/null**.

Os comandos vistos até o momento podem ser difíceis de lembrar, e dependendo da sua atuação, pode ser que seja interessante personalizar atalhos, comandos, variáveis de ambiente, entre outras coisas específicas para o seu caso de uso. Como estamos usando o shell bash, vamos comentar sobre seu arquivo de configuração, o **.bashrc**.

De forma simples, podemos dizer que toda vez que abrimos um terminal, ele executará o **.bashrc**. Portanto, se você precisar criar uma variável de ambiente, basta criá-la no arquivo **.bashrc**. Dessa forma, você não precisa digitar o comando no terminal a cada vez ou executar manualmente um script bash configurando o ambiente para as suas necessidades.

Por último, a Figura 11 mostra a estrutura de pasta do Linux de forma simplificada. O objetivo é ter uma ideia de onde procurar por algumas informações sobre o computador se necessário.

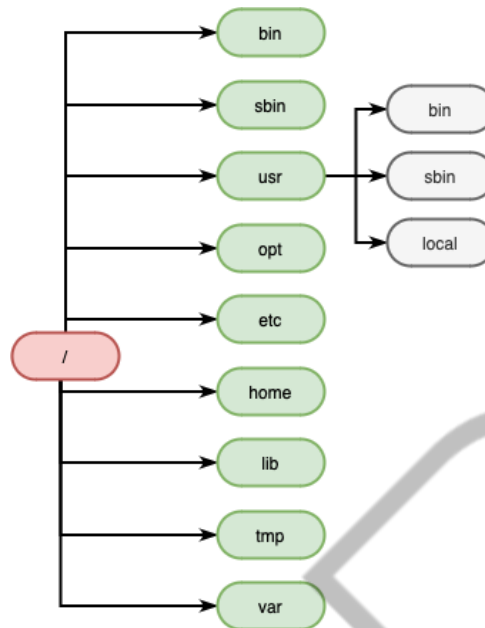


Figura 11 - Hierarquia de folders do Linux simplificada
Fonte: Elaborado pelo autor (2024)

Dependendo do SO podem ter mais ou menos folders. Entretanto, vamos explorar os mais comuns e interessantes para nosso contexto:

- bin (binaries): contém a maioria dos comandos de sistema que aprendemos nessa aula (ex: ls, cat, mv, cp, etc ...);
- sbin (super binaries): contém comandos que um usuário administrador utilizaria (não exploramos esses comandos);
- usr (user system resources): contém comandos de usuário. Por exemplo, se o usuário instalar um programa, ele será colocado neste folder;
- opt (optional): aplicativos opcionais são, normalmente, instalados aqui. Cada um em um subfolder próprio;
- etc: contém os arquivos de configurações dos programas e sistema;
- home: folder no qual normalmente trabalhamos (contém os usuários);
- lib (libraries): contém as bibliotecas compartilhadas pelos diferentes programas do sistema (provavelmente você não vai usar nada desse folder);
- root: home folder do usuário administrador (super user);
- tmp: contém arquivos temporários que são criados e removidos pelas aplicações sendo executadas no sistema;

- var: contém os logs das aplicações sendo executadas no sistema.

EMEND

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, tivemos um contato inicial com o conceito de shell. Vimos o que são variáveis de ambiente e porquê os comandos digitados no terminal funcionam. Aprendemos sobre os comandos e operadores Linux mais utilizados e sobre a hierarquia de pastas do Linux. Por fim, vimos o que é e como controlar o fluxo de erros utilizando um shell.



REFERÊNCIAS

MLOPS: pipelines de entrega contínua e automação no aprendizado de máquina. **Google Cloud**. 2023. Disponível em: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning?hl=pt-br>. Acesso em: 09 set. 2024.

PALEYES, A.; URMA, R, G.; LAWRENCE, N, D. Challenges in deploying machine learning: a survey of case studies. **ACM computing surveys**, v. 55, n. 6, p. 1-29, 2022.

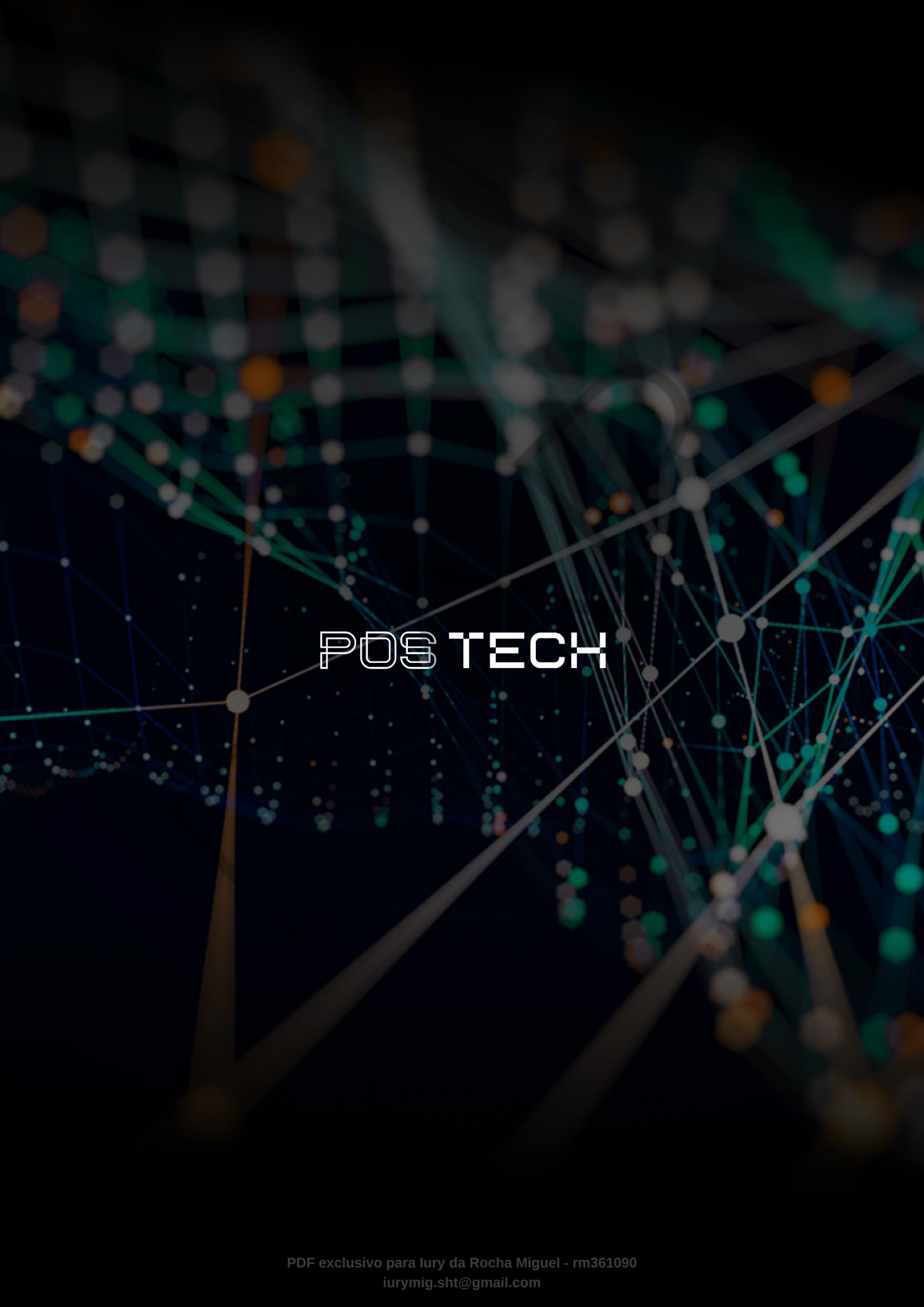
SATO, D. **Continuous Delivery for Machine Learning**. 2019. Disponível em: <https://martinfowler.com/articles/cd4ml.html>. Acesso em: 09 set. 2024.

TREVEIL, M. et al. **Introducing MLOps**. O'Reilly Media, 2020.

PALAVRAS-CHAVE

Palavras-chave: MLOps. Linux. Bash Programming.

EMEND



POSTECH